Classifying Fruits With a Decision Tree: A Step-by-Step Algorithm for the Kaggle Fruits Classification Dataset

Chittaranjan Rath

Indiana Wesleyan University

AIML-501-01R3I: Model Development (3FA2026)

Instructor: John Wishart

November 1, 2026

Classifying Fruits With a Decision Tree: A Step-by-Step Algorithm for the Kaggle Fruits Classification Dataset

Prediction sits at the center of machine learning. A model looks at examples it has already seen and uses them to predict something about an example it has not seen, whether that is the next word in a sentence or the type of fruit in a photograph. This paper does three things. It first describes, at a high level, how large language models (LLMs) make their predictions. It then examines the Kaggle Fruits Classification dataset (Saxena, 2023) and selects a decision tree as the method for classifying its fruits. Finally, it lays out the full classification process as plain-English, step-by-step instructions, including the decision points the tree uses, the parameter settings a practitioner would choose, the results that should be expected, and the reasons a decision tree fits this dataset better than K-means clustering. As the assignment allows, the process is described theoretically rather than executed in code, so the worked example and the results section describe what the method would do and how its output would be read rather than reporting numbers from a completed run.

How Large Language Models Work at a High Level

A large language model is a prediction machine whose single job is to guess the next piece of text. Everything else it appears to do, from answering questions to writing code, comes out of doing that one job extremely well over a very large amount of text (3Blue1Brown, 2024). The input text is first broken into tokens, which are words or word fragments, and each token is converted into a long list of numbers called an embedding. The embedding is a position in a high-dimensional space, and tokens with related meanings sit near one another in that space.

The embeddings then pass through a stack of transformer layers. Each layer contains an attention mechanism that lets every token look at every other token in the passage and pull in

context from the ones that matter, so the vector for the word bank moves toward the meaning of a riverbank when river appears nearby and toward a financial institution when loan does (Vaswani et al., 2017). Between the attention steps, feed-forward layers refine each vector further. After the final layer, the model turns the last vector into a probability for every token in its vocabulary and either picks the most likely one or samples from the distribution. That token is appended to the text and the whole process repeats, one token at a time (3Blue1Brown, 2024). The model learns the billions of numerical weights that drive this process by being shown enormous amounts of text and nudging the weights in whichever direction makes its next-token guesses less wrong.

The fruit classification problem in this paper is also a prediction problem, and it is a much smaller and more interpretable one. Where an LLM predicts one of tens of thousands of tokens from a context of thousands of numbers, the decision tree described below predicts one of five fruit labels from a handful of measurable attributes, and every step of its reasoning can be read directly off the tree.

Dataset Understanding

What the Dataset Contains

The Fruits Classification dataset on Kaggle is an image dataset rather than a spreadsheet. It contains 10,000 photographs of five fruits, with 2,000 images per class: apples, bananas, grapes, mangoes, and strawberries (Saxena, 2023). The images vary in shape, size, color, and lighting conditions, which is deliberate, since the dataset is meant for training and testing computer vision models for tasks such as fruit grading and sorting. The dataset is already divided into three folders. For each fruit, 97% of the images (1,940) are placed in a training folder, 2%

(40) in a validation folder, and 1% (20) in a test folder, and a small Python script that performs the split is included alongside the folders (Saxena, 2023). Table 1 summarizes the structure.

Table 1

Structure of the Kaggle Fruits Classification Dataset

Fruit class	Training images	Validation images	Test images	Total
Apple	1,940	40	20	2,000
Banana	1,940	40	20	2,000
Grape	1,940	40	20	2,000
Mango	1,940	40	20	2,000
Strawberry	1,940	40	20	2,000
Total	9,700	200	100	10,000

Note. Image counts per split follow the 97/2/1 division described on the dataset's data card (Saxena, 2023). Each image carries its label through the folder it sits in.

From Images to Measurable Attributes

The assignment asks for the dataset's characteristics, such as size, color, and weight. Because the raw data are pixels, those attributes are not stored as columns; they have to be measured from each photograph before a decision tree can use them. This is a normal preprocessing step for tree-based image classification, and it is the step that turns an image problem into a table problem. The label for every image is already known from its folder name, which is what makes the dataset suitable for a supervised method. Table 2 lists the attributes that would be extracted, how each would be measured, and which fruits it helps separate. Weight cannot be measured from a photograph, so it is left out; relative size in the frame is the closest available stand-in.

Table 2

Attributes Derived From Each Image and the Fruits They Help Separate

Attribute	How it is measured from the photo	Fruits it separates
Dominant hue	Convert the image from RGB to HSV, isolate the fruit from the background, and take the most common hue bin (eight bins around the color wheel)	Yellow bananas and mangoes from red apples and strawberries and from purple or green grapes
Aspect ratio	Longer side of the fruit's bounding box divided by the shorter side, so the value is always at least 1.0	Elongated bananas (about 2.5 to 3.5) and oval mangoes (about 1.3 to 1.6) from round apples and grapes (about 1.0 to 1.1)
Relative size	Area of the fruit region divided by the area of the whole image	Small grapes and strawberries from large apples and mangoes, with the caveat that close-up photographs inflate the value
Surface texture	Variation in brightness across the fruit region, scored with a Laplacian variance or local binary pattern measure	Seeded, dimpled strawberries and bumpy grape clusters from smooth apples, bananas, and mangoes
Color saturation	Mean of the saturation channel in HSV over the fruit region	Pale yellow bananas from deeply saturated red strawberries and orange mangoes
Fill ratio	Fruit area divided by bounding box area	Round, solid fruits (near 0.78) from irregular grape clusters and curved bananas (lower values)

Note. HSV refers to the hue, saturation, and value color space, which separates what color a pixel is from how bright it is.

Method Selection and Justification

How a Decision Tree Works

A decision tree classifies an item by asking a sequence of yes-or-no questions about its attributes. Each question is a node, each answer is a branch, and each end point is a leaf that carries a class label. To build the tree, the algorithm starts with all of the training examples in one group at the root and searches for the single question that divides them into the two purest possible sub-groups, meaning sub-groups whose members mostly share one label. It then repeats that search inside each sub-group and keeps going until the sub-groups are pure or a stopping

rule is reached (StatQuest with Josh Starmer, 2021). This recursive partitioning approach was formalized in the CART algorithm (Breiman et al., 1984) and, with a different purity measure, in ID3 (Quinlan, 1986).

Purity is measured with the Gini impurity. For a group in which a fraction p of the members belong to a given class, the Gini impurity is 1 minus the sum of the squared fractions across all classes. A group that contains only one fruit has an impurity of 0; a group in which five fruits are equally mixed has an impurity of 0.80. When the algorithm compares candidate questions, it computes the Gini impurity of each resulting sub-group, weights each by the share of examples it holds, and picks the question with the lowest weighted total (James et al., 2021, Chapter 8). In plain terms, the tree always asks the question that does the most to separate the fruits that are currently mixed together.

Why a Decision Tree Fits This Dataset

Four properties of the fruit dataset make a decision tree a strong fit. First, the data are labeled. Every one of the 10,000 images carries its fruit name in its folder, so a supervised method can use those labels directly during training, and the quality of the result can be measured exactly on the held-out test folder.

Second, the five fruits differ in ways that line up with simple threshold questions. A banana is far more elongated than anything else in the set, a strawberry is the only fruit with a visibly seeded surface, grapes are far smaller than apples or mangoes, and a mango is oval where an apple is round. Each of these is a single yes-or-no question on a single attribute, which is exactly the kind of rule a tree learns.

Third, the attributes are on different scales and of different types. Hue is a category, aspect ratio is a number between about 1 and 3, and relative size is a fraction between 0 and 1. A

tree asks about one attribute at a time, so it is unaffected by scale and needs no normalization. Distance-based methods such as K-nearest neighbors (Intuitive Machine Learning, 2020) and K-means both compute distances across all attributes at once, so they require careful scaling before they give sensible results.

Fourth, the result is interpretable. The finished tree is a flowchart that a person on a fruit-packing line could follow by eye, and the assignment's requirement to document each logical decision point maps directly onto the nodes of the tree. Feature importance also comes out of the process without extra work: the attributes used near the top of the tree are the ones doing most of the separating.

Why K-Means Clustering Was Not Selected

K-means is an unsupervised method. It is told only the number of groups to form ($k = 5$ here), places five center points in the attribute space, assigns each image to its nearest center, moves each center to the average of its members, and repeats until the assignments stop changing (MacQueen, 1967). It never looks at the labels. That creates two problems for this dataset. The clusters it finds follow whatever attribute dominates the distance calculation, which for fruit photos is usually color, so a green apple, a green grape, and a green mango could land in the same cluster while red and green apples are split apart. And even when the clusters are sensible, nothing in the method names them; a person still has to inspect each cluster and decide which fruit it represents. K-means is a good tool for discovering structure when labels are missing. Here the labels already exist, so throwing them away would discard the most useful information in the dataset.

Step-by-Step Classification Process

The process has four phases: preparing the data, growing the tree, stopping and pruning, and classifying a new fruit. The steps are written so that a reader without a machine learning background can follow them, with the technical name of each operation in parentheses for a reader who wants to implement it. A small worked example with ten hypothetical fruits accompanies the tree-growing phase so that the arithmetic behind each decision point is visible.

Phase 1: Preparing the Data (Preprocessing)

1. Load the images folder by folder, and record each image's label from its folder name (apple, banana, grape, mango, or strawberry). Keep the dataset's own train, valid, and test split rather than reshuffling, so that results are comparable with other work on the same data.
2. Resize every image to a common size, such as 224 by 224 pixels, so that size-based attributes are measured on the same footing.
3. Separate the fruit from the background (segmentation). Convert to HSV, apply a threshold on saturation to drop low-saturation backgrounds such as white plates or grey tables (Otsu's method picks the threshold automatically), and keep the largest connected region as the fruit. For cluttered backgrounds, a GrabCut pass initialized with the central region of the image works better. The OpenCV library provides all of these operations (Bradski, 2000).
4. Measure the six attributes in Table 2 on the fruit region and write one row per image into a table, with the label in the final column. At the end of this step the 10,000 images have become a table of 10,000 rows and seven columns.

5. Encode the one categorical attribute. Dominant hue becomes a set of yes-or-no columns, one per hue bin, so that the tree can ask whether the dominant hue is red as a single question (one-hot encoding).
6. Check the class balance. The dataset has exactly 1,940 training images per fruit, so no reweighting is needed. If a future version were unbalanced, the tree would be told to weight the rarer classes more heavily (class_weight set to balanced).

Phase 2: Growing the Tree

Steps 7 through 10 are the heart of the method. They are illustrated with the ten hypothetical fruits in Table 3, two of each class, whose attribute values are chosen to be realistic for the kinds of photographs in the dataset. The values are illustrative; on the real data the same steps run over 9,700 rows and the thresholds are learned from the measurements.

Table 3

Hypothetical Attribute Values for Ten Training Fruits Used in the Worked Example

No.	Fruit	Dominant hue	Aspect ratio	Relative size	Seeded surface
1	Apple	Red	1.05	0.70	No
2	Apple	Green	1.00	0.65	No
3	Banana	Yellow	2.80	0.80	No
4	Banana	Yellow	3.10	0.75	No
5	Grape	Purple	1.10	0.15	No
6	Grape	Green	1.00	0.20	No
7	Mango	Orange	1.50	0.90	No
8	Mango	Green	1.40	0.85	No
9	Strawberry	Red	1.20	0.25	Yes
10	Strawberry	Red	1.30	0.30	Yes

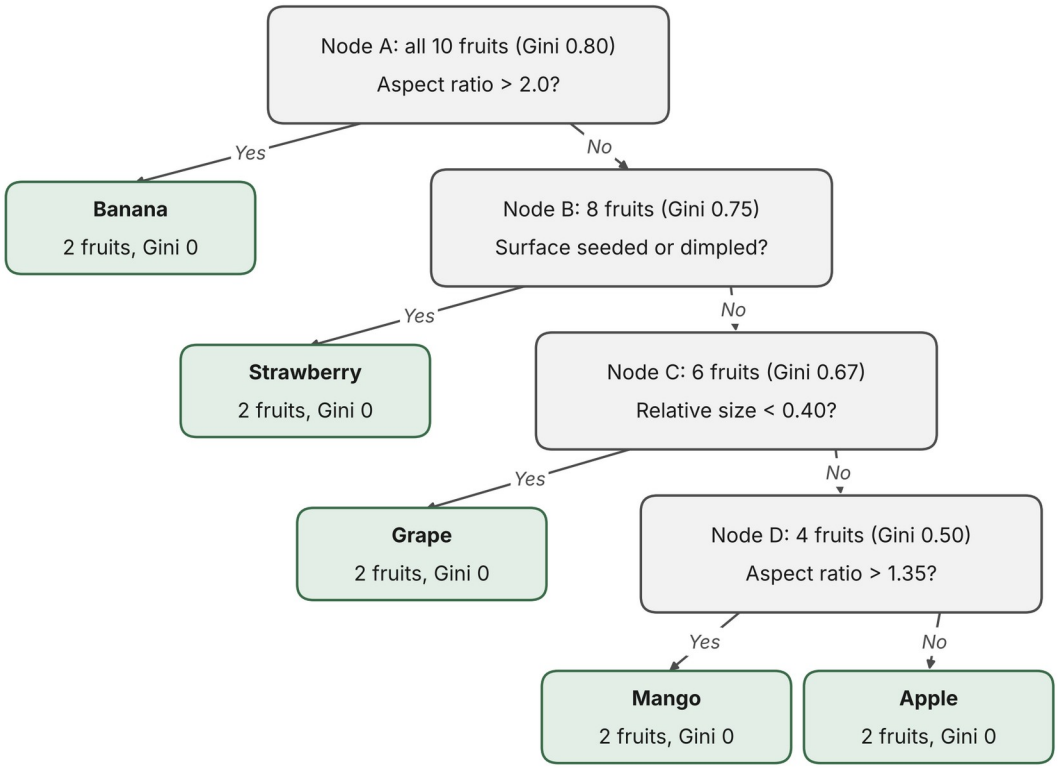
Note. Values are hypothetical and chosen to illustrate the method; they are not measurements taken from the Kaggle images.

7. Put all training fruits in one group at the root and measure how mixed it is. With two of each of five fruits, each class is 20% of the group, so the Gini impurity is $1 - 5 \times (0.20)^2 = 0.80$. This is the number the first question has to reduce.
8. List every candidate question. For each numeric attribute, the candidates are of the form "is the value greater than t?" for every threshold t that falls between two neighboring observed values; for each hue column, the candidate is "is the dominant hue this color?" Three of the candidates at the root are worked out in Table 4.
9. Score each candidate by the weighted Gini impurity of the two groups it produces, and keep the lowest. In Table 4, "aspect ratio greater than 2.0" sends the two bananas to one side (impurity 0) and the other eight fruits to the other side (impurity 0.75), for a weighted total of $0.2 \times 0 + 0.8 \times 0.75 = 0.60$. "Dominant hue is red" scores 0.65 because it mixes one apple in with the two strawberries and leaves the other apple on the far side. The seeded-surface question also scores 0.60, since it lifts out the strawberries just as cleanly. Ties like this are common when classes are balanced, because any question that removes one whole class produces the same total; the algorithm simply takes the first of the tied questions it evaluated (scikit-learn breaks ties by the order in which it considers the attributes; Pedregosa et al., 2011). This example takes aspect ratio, which becomes Node A in Figure 1, and the bananas become the first leaf.
10. Repeat Steps 8 and 9 inside each new group. The eight non-banana fruits (impurity 0.75) are split by "is the surface seeded?" into a pure strawberry leaf and a group of six with impurity 0.67 (Node B). Those six are split by "is the relative size less than 0.40?" into a pure grape leaf and a group of four with impurity 0.50 (Node C). The four remaining fruits are split by "is the aspect ratio greater than 1.35?" into a pure mango leaf and a

pure apple leaf (Node D). After four questions every leaf is pure, and the tree in Figure 1 is complete.

Figure 1

Illustrative Decision Tree Grown From the Ten Fruits in Table 3



Note. Grey boxes are questions (internal nodes) and green boxes are leaves. Each leaf shows the fruits that reached it and its Gini impurity. The thresholds come from the hypothetical values in Table 3 and would be re-learned from real extracted attributes.

Table 4

Gini Scores for Three Candidate Root Questions on the Worked Example

Candidate question	"Yes" group (fruits, Gini)	"No" group (fruits, Gini)	Weighted Gini	Decision
Aspect ratio greater than 2.0?	2 bananas (0.00)	2 apples, 2 grapes, 2 mangoes, 2 strawberries (0.75)	0.60	Selected (tied best)
Surface seeded?	2 strawberries (0.00)	2 apples, 2 bananas, 2 grapes, 2 mangoes (0.75)	0.60	Tied best; used at Node B
Dominant hue red?	1 apple, 2 strawberries (0.44)	1 apple, 2 bananas, 2 grapes, 2 mangoes (0.73)	0.65	Rejected

Note. Weighted Gini = (share of fruits in the yes group × its Gini) + (share in the no group × its Gini). Lower is better. The root impurity before any question is 0.80.

Phase 3: Stopping and Pruning

11. Stop splitting a group when any of the following holds: it is already pure; it holds fewer fruits than the minimum allowed for a split (min_samples_split); splitting it would create a leaf smaller than the minimum leaf size (min_samples_leaf); or the group already sits at the maximum depth (max_depth). On the ten-fruit example the tree stops because every leaf is pure. On the real 9,700-image training set it would stop on the size and depth rules well before every leaf is pure, which is intentional.
12. Label every leaf with the majority fruit among the training images that reached it. The share of that majority is the confidence the tree will report for new fruits that land there.
13. Prune the tree using the validation folder (cost-complexity pruning). Grow a deliberately large tree, then remove the branches that improve training accuracy the least, and keep the version whose accuracy on the 200 validation images is highest. This guards against the tree memorizing quirks of individual photographs, such as a shadow that happened to fall on one apple, instead of learning rules that generalize.

Phase 4: Classifying a New Fruit

14. Take a new photograph, run it through Steps 2 to 5 to produce its row of attribute values, and start at the root.
15. Answer the root question from the row, follow the yes or no branch, answer the next question, and continue until a leaf is reached. The leaf's label is the prediction.

For example, a new image with a dominant hue of green, an aspect ratio of 1.45, a relative size of 0.88, and a smooth surface would be handled by the tree in Figure 1 as follows. Node A: 1.45 is not greater than 2.0, so take the no branch. Node B: the surface is not seeded, so take the no branch. Node C: 0.88 is not less than 0.40, so take the no branch. Node D: 1.45 is greater than 1.35, so take the yes branch. The image is classified as a mango, and the tree reached that answer without ever using color, which is what lets it handle a green mango correctly even though green is also an apple and grape color.

Parameter Settings

Table 5 lists the settings a practitioner would choose for a run on the full dataset, with the scikit-learn name of each (Pedregosa et al., 2011), its plain-English meaning, and the reason for the value.

Table 5

Parameter Settings for the Decision Tree on the Full Dataset

Parameter (scikit-learn name)	Value	Plain-English meaning and reason for the value
Purity measure (criterion)	gini	Score each candidate question by Gini impurity, as in Table 4. Entropy would give nearly the same tree and is slightly slower to compute.
Maximum depth (max_depth)	6	No fruit is asked more than six questions. Five classes with the attributes in Table 2 separate in three or four questions, so six leaves room for exceptions without memorizing individual photos.

Parameter (scikit-learn name)	Value	Plain-English meaning and reason for the value
Minimum fruits to split (min_samples_split)	40	A group must hold at least 40 training images before it can be split further, so no rule is learned from a handful of unusual photos.
Minimum leaf size (min_samples_leaf)	20	Every leaf must contain at least 20 training images, which keeps each leaf's majority label statistically meaningful.
Class weighting (class_weight)	None	Classes are balanced at 1,940 training images each, so no reweighting is needed.
Pruning strength (ccp_alpha)	Tuned on validation	Tried over a range of values; the value that gives the best accuracy on the 200 validation images is kept.
Random seed (random_state)	42	Makes tie-breaking between equally good questions repeatable from run to run.
Image size (preprocessing)	224 × 224 pixels	Common size before measuring attributes so that relative size and texture are comparable across images.
Hue bins (preprocessing)	8	Coarse enough that lighting shifts do not move a fruit between bins, fine enough to separate red, orange, yellow, green, and purple.

Note. Parameter names in parentheses are those used by the DecisionTreeClassifier in scikit-learn (Pedregosa et al., 2011).

Key Decision Points and How They Lead to the Final Classification

Table 6 collects the decision points of the method and states, for each, the criterion that drives it and how it moves a fruit toward its final label. The first three rows are decisions the algorithm makes while learning; the remaining rows are the questions the finished tree asks.

Table 6

Decision Points in the Method and Their Effect on the Final Classification

Decision point	Criterion used	How it leads to the final classification
Which question to ask at each node	Lowest weighted Gini impurity among all candidate attribute and threshold pairs	Each question removes as much class mixing as possible, so fruits are separated in the fewest steps
When to stop splitting	Purity reached, group below the minimum size, or maximum depth reached	Prevents rules that fit single photographs; leaves stay large enough that their majority label is reliable

Decision point	Criterion used	How it leads to the final classification
How much to prune	Highest accuracy on the validation folder across pruning strengths	Keeps only the branches that help on unseen images, which is what the test folder will measure
Node A: aspect ratio greater than 2.0	Bananas are the only strongly elongated fruit	Isolates bananas at the first question
Node B: surface seeded or dimpled	Strawberries are the only fruit with a seeded skin	Isolates strawberries from the remaining round and oval fruits
Node C: relative size less than 0.40	Grapes are far smaller in the frame than apples or mangoes	Isolates grapes from the two large fruits
Node D: aspect ratio greater than 1.35	Mangoes are oval; apples are round	Separates the last two classes, giving every image one of the five labels

Note. Node letters refer to Figure 1.

Expected Results and Evaluation

Metrics

Because the assignment is theoretical, no run was performed, and the numbers in this section are the ones the evaluation would produce rather than measured values. The evaluation itself follows a fixed protocol. The tree is grown on the 9,700 training images, pruned using the 200 validation images, and scored once on the 100 test images that it has never seen. Scoring on the test folder, and only once, is what makes the reported numbers an honest estimate of performance on new photographs (James et al., 2021, Chapter 5). Table 7 lists the metrics and how each would be read.

Table 7

Evaluation Metrics for the Decision Tree and How Each Is Interpreted

Metric	What it measures	How it would be read for the fruit tree
Accuracy	Share of the 100 test images given the correct label	The headline number; with five balanced classes, random guessing scores 0.20, so anything near 0.90 or above shows the attributes carry real signal

Metric	What it measures	How it would be read for the fruit tree
Confusion matrix	A 5 by 5 table of true label against predicted label	Shows which fruits are mistaken for which; the off-diagonal cells point to the attribute that needs improvement
Precision per class	Of the images the tree called apple, the share that were apples	Low precision for apple means other fruits are leaking into the apple leaf
Recall per class	Of the real apples, the share the tree caught	Low recall for apple means apples are being sent elsewhere, most likely down the strawberry branch
F1 score per class	Harmonic mean of precision and recall	A single balanced number for each fruit; the lowest F1 identifies the hardest class
Feature importance	How much each attribute reduced impurity across the whole tree	Confirms which attributes the tree relied on; the expected order is aspect ratio, surface texture, relative size, then hue
Training versus test accuracy gap	Difference between accuracy on the training and test folders	A large gap means overfitting and calls for stronger pruning or a smaller maximum depth

Note. Precision, recall, and F1 are computed separately for each of the five fruits and then averaged for a single summary.

Expected Patterns in the Results

On the worked example, all ten training fruits reach a pure leaf, so training accuracy is 100%. That figure is optimistic by construction, because the tree was allowed to keep splitting until every leaf was pure, and it is exactly why the test folder, not the training folder, is used for the reported accuracy. On the full dataset, four patterns can be expected from the way the attributes behave.

Bananas should be the easiest class, with precision and recall near 1.0, because their aspect ratio is far from every other fruit's and the root question captures it directly. Apples and strawberries share a red hue, so any tree that leans on color will confuse them; the texture and size questions are what keep them apart, and the apple and strawberry cells of the confusion matrix are where most of the remaining errors should sit, especially for close-up strawberry photos that fill the frame and so look large. Green mangoes and green apples are the second

likely confusion, resolved by the aspect ratio question at Node D, so mango recall will depend on how reliably the segmentation step recovers the fruit's outline. Grapes photographed as a tight cluster can read as one large, bumpy object; when that happens they may be sent down the large-fruit branch, which is the main reason the fill ratio attribute is included.

Taken together, these patterns mean the feature importance ranking should put aspect ratio and surface texture at the top and dominant hue lower than a person might expect. That is a useful insight about the dataset itself: its fruits are more reliably told apart by shape and skin than by color, largely because the photographs were taken under varied lighting.

Implementation Challenges and Solutions

Table 8 lists the problems a practitioner would meet while carrying out the steps above, why each arises in this particular dataset, and the solution built into the process.

Table 8

Implementation Challenges and Their Solutions

Challenge	Why it arises in this dataset	Solution
Cluttered or non-white backgrounds	Images come from varied sources, so the fruit is not always on a plain background	Segment on saturation first and fall back to GrabCut when the largest region is less than 10% of the image
Lighting variation shifts colors	The data card states that lighting varies across images	Measure hue in HSV, which separates color from brightness, and use coarse hue bins
Photo scale makes relative size unreliable	A close-up strawberry can occupy more of the frame than a distant apple	Pair relative size with fill ratio and texture so that no single question depends on scale alone
Overfitting to individual photos	A fully grown tree on 9,700 images can carve out leaves for single oddities	Minimum leaf size of 20, maximum depth of 6, and validation-based pruning
Ties between equally good questions	Balanced classes make several questions score the same, as in Table 4	Fix the random seed so the tie-break is repeatable, and document the tree that results
Grape clusters measured as one object	Segmentation returns the whole bunch rather than one grape	Treat the cluster as the object and rely on fill ratio and texture, which are distinctive for a bunch

Note. GrabCut is an interactive foreground extraction method available in OpenCV (Bradski, 2000).

Conclusion

The Kaggle Fruits Classification dataset is a labeled collection of 10,000 photographs of five fruits. Once each photograph is reduced to a short row of measurable attributes, a decision tree classifies it by asking a short sequence of yes-or-no questions, each chosen because it does the most to separate the fruits still mixed together at that point. On the worked example, four questions on aspect ratio, surface texture, relative size, and aspect ratio again are enough to give every fruit its own leaf, and a new green mango is classified correctly without the tree ever consulting color. The method fits this dataset because the labels exist, the fruits differ along simple thresholds, the attributes need no scaling, and the finished tree is a flowchart anyone can follow. K-means would discard the labels and produce groups that still need naming, which is why it was not chosen. The main risks, background clutter, lighting, and overfitting, each have a standard remedy in the preprocessing and pruning steps, and the test folder provides an honest measure of how well those remedies worked.

References

3Blue1Brown. (2024, April 1). *Transformers, the tech behind LLMs | Deep learning chapter 5* [Video]. YouTube. https://www.youtube.com/watch?v=wjZofJX0v4M

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools, 25*(11), 120–125.

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth.

Intuitive Machine Learning. (2020, April 26). *K nearest neighbors | Intuitive explained | Machine learning basics* [Video]. YouTube. https://www.youtube.com/watch?v=0p0o5cmgLdE

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer. https://doi.org/10.1007/978-1-0716-1418-1

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In L. M. Le Cam & J. Neyman (Eds.), *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 281–297). University of California Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1*(1), 81–106. https://doi.org/10.1007/BF00116251

Saxena, U. (2023). *Fruits classification* [Data set]. Kaggle. https://www.kaggle.com/datasets/utkarshsaxenadn/fruits-classification

StatQuest with Josh Starmer. (2021, April 25). *Decision and classification trees, clearly explained!!!* [Video]. YouTube. https://www.youtube.com/watch?v=_L39rN6gz7Y

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*, 5998–6008.